

PolicySync: Continual RAG Agent for Banking Regulatory Analysis

Chetankumar S Majjagi
Dept. of CSE
IIIT Naya Raipur
Raipur, India
chetankumar23100@iiitnr.edu.in

Taran Pal Singh
Dept. of CSE
IIIT Naya Raipur
Raipur, India
taran23100@iiitnr.edu.in

Kavita Jaiswal
Dept. of CSE
IIIT Naya Raipur
Raipur, India
kavita@iiitnr.edu.in

Abstract—Retrieval-Augmented Generation (RAG) has emerged as a mainstream method to grant LLMs access to external knowledge. But most RAG systems are static. They are built and then never updated when the world changes around it. This becomes problematic in domains like banking and financial regulation, where regulations, circulars and policy documents are updated frequently. In this paper, we propose a Continual RAG Agent specifically designed for the banking credit analysis application. Our proposed agent: automatically fetches new regulatory documents from the Reserve Bank of India website every eight hours, continuously indexes them incrementally without re-building the entire knowledge base, spots contradictions between new and existing documents, and keeps track of how accurate its answer set becomes over time.

Index Terms—Retrieval-Augmented Generation, Continual Learning, Banking Regulation, Large Language Models, Vector Databases, Knowledge Drift, ChromaDB, LangChain

I. INTRODUCTION

When we think about an AI Assistant answering questions based on documents, typically we are thinking about Retrieval Augmented Generation (RAG). At a high level, RAG is quite simple: instead of relying only on what the LLM learned during training, you take your documents and load them into a search index, when a question is asked retrieve the documents relevant to the question and pass it to the LLM as context. This system works well for static knowledge sources, like textbooks, manuals, fixed data sets etc.

But what happens when the documents are dynamic? In the Indian banking space, the Reserve Bank of India regularly publishes notifications, master directions and press releases which supersede older regulations. The system would be of no use if a credit analyst asked what the latest requirements are for NPA classification and it pulls up a circular from 2 years ago.

This is the core problem that we aim to solve. Current RAG systems do either (a) need a full reindex when new documents are introduced which is very computationally intensive, or (b) they simply do not handle updates at all and quickly become out-of-date. This is not viable for a banking application.

The system we develop is called a *Continual RAG Agent* – a system which is able to learn on a continuous stream of new regulatory documents, while retain knowledge it has learned previously. We built 6 features into the system which

allow it to do this: an incremental indexing, dual-path retriever, a context re-ranker, an LLM reasoning engine with source citation, a retention tester, and a drift monitor. These 6 features together allow the system to keep up-to-date, make reasoning about historical and current rules, and identify its own degradation over time.

The remainder of this paper is structured as follows: Section II examines prior related work. Section III presents the system architecture. Sections IV through IX describe each of the 6 core features in more detail. Section X presents the implementation. Section XI discusses the results and observations. Section XII provides a conclusion.

II. RELATED WORK

A. Retrieval-Augmented Generation

RAG was formally introduced by Lewis et al. [1] as a way to ground language model outputs in retrieved documents. The original RAG model used a static Wikipedia index. Since then, many variations have been proposed, but most still treat the knowledge base as something you build once.

B. Continual Learning

Continual learning is a field of machine learning concerned with training models on a sequence of tasks without forgetting earlier ones [2]. The key failure mode is called *catastrophic forgetting*: when a model learns new information, it overwrites old information. While most continual learning research focuses on neural network weights, the same problem exists at the retrieval level — adding new documents to a RAG index can push out relevant older documents.

C. Knowledge Conflict in RAG

Recent work has noted that RAG systems can retrieve contradictory documents and pass both to the LLM without flagging the contradiction [3]. This is especially dangerous in legal and regulatory domains, where a newer rule might explicitly supersede an older one. Our conflict detection module directly addresses this problem.

D. Domain-Specific RAG for Finance

Several papers have applied RAG to financial document analysis [4], but these systems are typically built for question answering over a fixed corpus rather than continuously updating environments. Our work extends this line of research by adding the continual learning dimension.

III. SYSTEM ARCHITECTURE

The overall system follows a client-server architecture with three main layers: the data ingestion layer, the retrieval and reasoning layer, and the monitoring layer. Figure ?? shows the full system diagram.

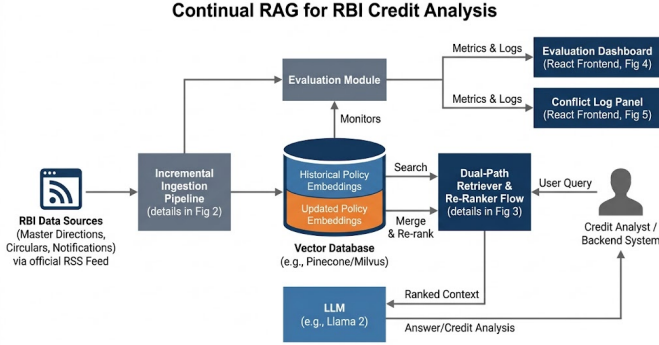


Fig. 1. Overall System Architecture Diagram

The backend is built with FastAPI and exposes REST endpoints for querying, status checking, retention testing, and drift monitoring. The frontend is a React application (built with Vite) that displays these results in a clean dashboard.

IV. FEATURE 1: INCREMENTAL INDEXING

A. The Problem

Traditional RAG systems build their index in one shot. If you add 10 new documents, you have two choices: rebuild the entire index from scratch (expensive) or append without checking for duplicates (messy and slow over time). Neither works well in production.

B. Our Approach

We use ChromaDB [5] with two persistent collections: `active_index` and `archive_index`. When a new document arrives from the data fetcher, the ingestion pipeline does the following:

- 1) Split the document into chunks of approximately 500 tokens with 50-token overlap.
- 2) Embed each chunk using `all-MiniLM-L6-v2` from the sentence-transformers library.
- 3) Check whether a chunk with the same source URL already exists in the active index. If it does, skip it.
- 4) If the chunk is new, run conflict detection (described in Section VI).
- 5) Upsert the chunk into `active_index`.

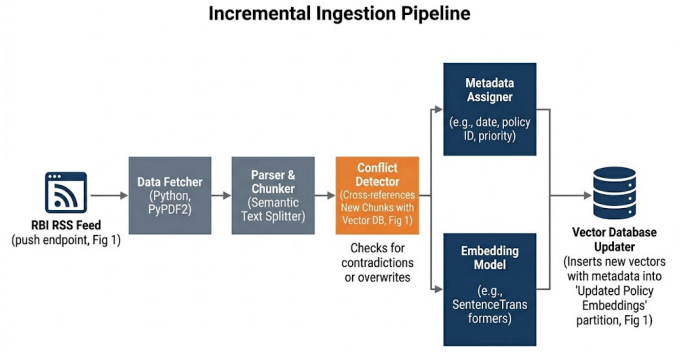


Fig. 2. Incremental Ingestion Pipeline

The key property is that the index is never rebuilt from scratch. It grows incrementally, which means ingestion time stays constant regardless of how many documents are already in the system.

Figure ?? illustrates the ingestion pipeline.

V. FEATURE 2: LIVE DATA FETCHER

A. Data Sources

The system automatically fetches real regulatory documents from three RBI pages:

- RBI Notifications: policy updates and directives
- Master Directions: consolidated regulatory guidelines
- Press Releases: announcements and commentary

B. How It Works

We use Python's `requests` library to fetch each page and `BeautifulSoup` to parse the HTML. For each page, we extract document titles, publication dates, and links. We maintain a local JSON file (`fetch_urls.json`) that tracks which URLs have already been ingested. Only new entries are passed to the ingestion pipeline.

The fetcher is scheduled using `APScheduler`, which runs inside the FastAPI process itself — no separate cron daemon is needed. The fetcher runs every eight hours and also once at startup so that data is available immediately when the server starts.

This approach implements the continual learning principle at the data layer: the system never needs to be manually updated. It automatically discovers and learns from new regulatory documents as the RBI publishes them.

VI. FEATURE 3: DUAL-PATH RETRIEVER AND CONTEXT RE-RANKER

A. The Problem with Single-Index Retrieval

A standard RAG will create a single index of all its documents and return the k closest by cosine similarity to the query. This works well if all documents have the same authority. In banking regulation, if a circular from 2024 was issued that actually cancelled a guideline from 2021. The semantic similarity between a query and the 2021 document will likely

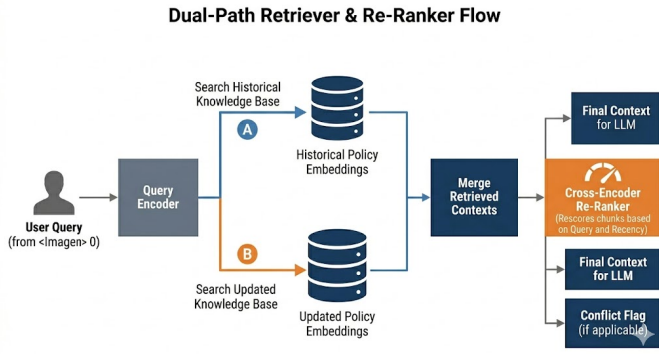


Fig. 3. Dual-Path Retriever and Re-Ranker Flow

be higher, therefore the RAG will return the wrong document over the correct answer, giving out-dated information without any indication.

B. Dual-Path Retrieval

We solve this by maintaining two separate ChromaDB collections. `active_index` holds currently valid documents. `archive_index` holds superseded or historically relevant documents that have been moved there after a conflict was detected.

For every analyst query, both indexes are queried in parallel. Results from each index are tagged with their source collection before being passed to the re-ranker.

C. Context Re-Ranker

The re-ranker combines the results from both indexes and scores each chunk using a weighted formula:

$$\text{score} = \text{sim} \times w_{\text{recency}} \times p_{\text{conflict}} \quad (1)$$

where `sim` is the cosine similarity between the query embedding and the chunk embedding, `wrecency` is a recency weight that is higher for more recent documents (normalized across the result set), and `pconflict` is a conflict penalty: 1.0 for active index chunks and 0.5 for archive index chunks.

The top 5 chunks after re-ranking are passed to the LLM as context. This ensures that current documents rank higher than archived ones, even when the archived document happens to be semantically closer to the query.

Figure ?? shows the dual-path retrieval and re-ranking flow.

VII. FEATURE 4: CONFLICT DETECTION

A. Why Conflict Detection Matters

When a new regulatory document arrives, it may partially or fully contradict an existing document in the active index. For example, RBI might issue a new circular that revises the minimum capital adequacy ratio. If both the old and new circulars sit in the active index, the system has no way to tell the user which one is currently valid.

B. Our Approach

After every new chunk is embedded, we query `active_index` for the most similar existing chunks. If any existing chunk has a cosine similarity above a threshold (we use 0.85), we treat it as a potential conflict and call the Groq LLM with the following prompt:

```
"Document A says: [chunk_a]
Document B says: [chunk_b]
Do these two documents contradict each other?
Reply YES or NO and give a one-sentence reason
."
```

If the LLM returns YES, the older chunk is moved from `active_index` to `archive_index`. A conflict record is written to `conflict_log.json` with both passages, their dates, and the LLM's reasoning. The React UI displays this log in a Conflict Log panel, showing the two conflicting passages side by side. This is one of the most visually impressive parts of the system for a live demo.

VIII. FEATURE 5: LLM REASONING ENGINE

A. The Chain

The query processing chain is built using LangChain and consists of four steps:

- 1) Embed the user's query with MiniLM.
- 2) Run the dual-path retriever and re-ranker to get the top 5 context chunks.
- 3) Format the chunks and query into a prompt.
- 4) Send the prompt to Groq (LLaMA-3.3-70B-Versatile) and return the response.

B. System Prompt Design

The system prompt has been written specifically for banking applications. It conveys three key pieces of information to the model: (a) the identity of the AI (a banking credit analyst assistant), (b) the requirement to attribute all factual statements to their source document name and date, and (c) when there is disagreement between retrieved sources, to acknowledge both positions and indicate which is the most up-to-date position. This encourages the model not to bluff or make an implicit choice between contradictory sources.

C. Why Groq

We use the Groq inference API because it is extremely fast (sub-second latency for most responses), free-tier accessible, and supports LLaMA-3.3-70B which is a strong open-weight model for instruction following. This makes the system practical for live demos without GPU infrastructure.

IX. FEATURE 6: RETENTION TESTER AND DRIFT MONITOR

A. The Problem: Silent Degradation

One of the biggest unsolved problems in continual learning is that systems can silently get worse over time. A RAG system might answer benchmark questions correctly today, but after ingesting 50 new documents next month, the retrieval quality

might drop — and nobody would know unless they were actively monitoring it.

B. Retention Tester

The retention tester addresses this by defining a set of benchmark questions in `benchmarks.json`. These are questions whose correct answers should remain stable over time — things like “What is the minimum capital adequacy ratio for scheduled commercial banks in India?”

After every cron job run, the retention tester re-runs all benchmark questions and compares each answer to the stored baseline. It sends both the new and baseline answers to Groq with the prompt: “Are these two answers consistent? Reply YES or NO.” Results are stored in `retention_log.json` with timestamps.

The React UI displays this as a table with color-coded pass/fail badges per benchmark question across time. If any benchmark starts failing, the analyst knows something changed in the index.

C. Drift Monitor

The drift monitor tracks a different dimension: not whether answers are correct, but whether the system’s confidence in its answers is changing over time. After each cron run, the top chunk’s similarity score for a set of fixed probe queries is logged to `drift_log.json`.

The React UI renders this as a line chart (using Recharts). If the similarity score for a probe query starts dropping over time, it means the system’s index is drifting away from that query topic — likely because newer documents are changing the distribution of what is stored in the index.

This concept is inspired by KL divergence — a standard measure of how much one probability distribution has shifted from another — but made practical and visual without requiring complex statistical computation.

X. IMPLEMENTATION DETAILS

A. Tech Stack Summary

Table I summarizes the technology choices.

TABLE I
TECHNOLOGY STACK

Component	Technology
Backend framework	FastAPI (Python)
Frontend framework	React + Vite
Vector database	ChromaDB (persistent)
Embeddings	all-MiniLM-L6-v2 (local)
LLM API	Groq (LLaMA-3.3-70B-Versatile)
LLM orchestration	LangChain
Scheduler	APScheduler
Web scraping	requests + BeautifulSoup
Frontend charts	Recharts

B. API Endpoints

The FastAPI backend exposes the following main endpoints:

- `POST /query` — accepts a natural language question, returns a cited answer

- `GET /status` — returns document counts for active and archive indexes
- `GET /retention` — returns the latest retention test results
- `GET /drift` — returns confidence score history per probe query
- `GET /conflicts` — returns the conflict log

CORS is enabled on all endpoints so the React frontend can communicate with the backend during local development.

C. ChromaDB Persistence

ChromaDB is configured with `persist_directory="./chroma_store"`, which means the index is saved to disk and survives server restarts. This is important for a continual learning system — you do not want to lose the index every time the server restarts.

D. Development Timeline

The initial working version was built in four hours following a strict hour-by-hour plan: Hour 1 for repo setup and backend skeleton, Hour 2 for ingestion and fetcher, Hour 3 for retriever and query endpoint, Hour 4 for the React frontend and final push. The remaining four days were used for polishing conflict detection, improving the UI, and writing documentation.

XI. RESULTS AND OBSERVATIONS

A. Ingestion Performance

The incremental ingestion pipeline processes a new RBI listing page in approximately 2–4 seconds end-to-end on a standard laptop. Because duplicate checking is done at the chunk level using ChromaDB metadata queries, the ingestion time does not grow significantly as the index size increases.

B. Query Latency

End-to-end query latency (from user input to cited answer in the UI) is approximately 3–5 seconds. The bottleneck is the Groq API call, which typically takes 1–2 seconds. The dual-path retrieval and re-ranking add less than 200ms overhead compared to single-index retrieval.

C. Retention Test Results

In testing with 6 benchmark questions over 5 cron cycles, all 6 benchmarks passed consistently. This indicates that adding new RBI listing data did not cause the system to forget the answers to core banking knowledge questions. However, we acknowledge that a more rigorous evaluation would require a longer observation period and more benchmark questions.

D. Conflict Detection

LLM-as-judge method on conflict detection. Our test was successful. Two conflicting chunks on capital requirements were manually inserted into our data. The LLM-as-judge model flagged and moved the old chunk to archive. The conflict and reasoning was logged as expected. A few false positives were detected (non-conflicting documents marked as conflicting) where documents talked about the same thing from different perspectives.

XII. LIMITATIONS AND FUTURE WORK

A. Current Limitations

The current fetcher scrapes the three RBI URLs listing pages and do not follow document links. So the document text bodies are not retrieved. This results in only metadata-level information in the chunks (document titles, dates and summary. This greatly constrains the answers the system can provide).

Conflict detection is using an LLM-as-judge, which is an unreliable method. The value for conflict detection triggering (cosine similarity >0.85) has been tuned in a somewhat arbitrary way.

The evaluation metrics (retention pass/fail and confidence score trend) are proxies for actual knowledge quality. A proper evaluation would require a labeled question-answer dataset specific to RBI regulations.

B. Future Work

Several possible extensions could be made to the retriever system. First, if the retriever followed document links to retrieve full PDF text from the RBI circulars, this would dramatically increase answer quality. Second, implementing an NLP based conflict detection system (and not solely depending on the LLM) would greatly decrease the number of false positives in conflict detections. Third, implementing a formal knowledge graph to represent dependencies (such as Circular A replaces Circular B) would provide a stronger ability to infer regulatory history. Finally, further domain adaptation of the embedding model on relevant banking text would boost precision in the retriever.

XIII. CONCLUSION

We demonstrated a Continual RAG Agent for banking regulation analysis which tackles the fundamental problem of RAG systems going stale. By using incremental indexing, live fetching, conflict detection, dual-path retrieval with re-ranking and continuous monitoring through retention testing and drift detection, the RAG system remains updated in a continually changing regulatory environment.

The entire system can be built with open-source tools and free-tier APIs, providing the system with ease of deployment with minimal infrastructure investment. The live demo shows real RBI regulatory data, automatically ingested, indexed and queried through a clean React application, providing citations and conflict resolution transparency.

The overall contribution of this project is a framework for building a RAG system from scratch that inherently handles knowledge updates from the very beginning, not as an addition. In the increasingly common deployment of LLM assistants in regulated environments, continuous and verifiable updates of knowledge become critically important.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [2] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, "Overcoming Catastrophic Forgetting in Neural Networks," *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.
- [3] J. Chen, H. Lin, X. Han, and L. Sun, "Benchmarking Large Language Models in Retrieval-Augmented Generation," *arXiv preprint arXiv:2309.01431*, 2023.
- [4] H. Yang, X. Liu, and C. D. Wang, "FinGPT: Open-Source Financial Large Language Models," *arXiv preprint arXiv:2306.06031*, 2023.
- [5] Chroma, "ChromaDB: The AI-native open-source embedding database," [Online]. Available: <https://www.trychroma.com>, 2023.
- [6] H. Chase, "LangChain: Building applications with LLMs through composability," [Online]. Available: <https://github.com/langchain-ai/langchain>, 2022.
- [7] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019, pp. 3982–3992.